Actividad evaluativa 1
Centro de monitoreo de accesos

Alex Villegas
Programación Orientada a Objetos – 2.º de Ciberseguridad

Octubre de 2026

1. Objetivo

En esta actividad he hecho un programa en Java que analiza los intentos de acceso a dos servidores. El programa comprueba si los datos introducidos son válidos, calcula el porcentaje de intentos fallidos y clasifica cada servidor como NORMAL, ALERTA o BLOQUEADO. Finalmente, indica cuál de los dos necesita más atención.

2. Funcionamiento del programa

Todo el código está en el archivo `Main.java`. He utilizado dos clases:

- `Servidor`: guarda los datos de cada servidor y tiene los métodos para validarlos, calcular el porcentaje, evaluar el estado y mostrar el resumen.
- `Main`: crea los dos servidores, pide los datos por teclado con `Scanner` y compara sus prioridades.

Primero se comprueban los datos. El nombre no puede estar vacío, el origen debe ser interno o externo, la criticidad debe ser si o no, el total de intentos debe ser mayor que cero y los intentos fallidos deben estar entre cero y el total. Para los textos se utiliza `trim()` y `equalsIgnoreCase()`.

Si los datos son incorrectos, el estado es `DATOS INVALIDOS` y no se calcula el porcentaje. Si son correctos, se aplican las reglas en este orden:

1. `BLOQUEADO`: cinco o más fallos, o servidor crítico con origen externo y al menos tres fallos.
2. `ALERTA`: tres o cuatro fallos, o al menos dos fallos si el origen es externo, siempre que no esté bloqueado.
3. `NORMAL`: si no se cumplen las condiciones anteriores.

Para comparar los servidores, he asignado una prioridad numérica a cada estado: 3 para BLOQUEADO, 2 para ALERTA y 1 para NORMAL. Si alguno tiene datos inválidos, no se realiza la comparación.

3. Pruebas del programa

He organizado los nueve casos del enunciado en cinco ejecuciones. En cada ejecución introduzco los datos de dos servidores y hago una captura de la terminal con el resultado. La última ejecución la he aprovechado también para comprobar los espacios y las mayúsculas.

3.1. Prueba 1: casos 1 y 2

- Servidor 1: VPN, externo, si, 10 intentos, 3 fallidos. Resultado esperado: `BLOQUEADO`.
- Servidor 2: WEB, interno, no, 8 intentos, 4 fallidos. Resultado esperado: `ALERTA`.
- Comparación esperada: prioridad para VPN.

```
PS C:\Users\alexv\Desktop\Uni\2o curso\Programación Orientada a Objetos\Actividad1_POO_AVS> java Main
Nombre del servidor 1: VPN
Origen (interno/externo): externo
Critico (si/no): si
Intentos totales: 10
Intentos fallidos: 3
Nombre del servidor 2: WEB
Origen (interno/externo): interno
Critico (si/no): no
Intentos totales: 8
Intentos fallidos: 4

========== REPORTE DE SEGURIDAD ==========
VPN | 3/10 | 30.0% | BLOQUEADO
WEB | 4/8 | 50.0% | ALERTA
ATENCIÓN PRIORITARIA: VPN
```

Figura 1: Prueba 1: casos 1 y 2.

3.2. Prueba 2: casos 3 y 4

- Servidor 1: CORREO, externo, no, 8 intentos, 2 fallidos. Resultado esperado: `ALERTA`.
- Servidor 2: VPN, interno, si, 12 intentos, 2 fallidos. Resultado esperado: `NORMAL`.
- Comparación esperada: prioridad para CORREO.

```
PS C:\Users\alexv\Desktop\Uni\2o curso\Programación Orientada a Objetos\Actividad1_POO_AVS> java Main
Nombre del servidor 1: CORREO
Origen (interno/externo): externo
Critico (si/no): no
Intentos totales: 8
Intentos fallidos: 2
Nombre del servidor 2: VPN
Origen (interno/externo): interno
Critico (si/no): si
Intentos totales: 12
Intentos fallidos: 2

========== REPORTE DE SEGURIDAD ==========
CORREO | 2/8 | 25.0% | ALERTA
VPN | 2/12 | 16.666666666666668% | NORMAL
ATENCIÓN PRIORITARIA: CORREO
```

Figura 2: Prueba 2: casos 3 y 4.

3.3. Prueba 3: casos 5 y 6

- Servidor 1: WEB, externo, no, 5 intentos, 5 fallidos. Resultado esperado: `BLOQUEADO`.
- Servidor 2: WEB, interno, no, 4 intentos, 5 fallidos. Resultado esperado: `DATOS INVALIDOS`.
- Comparación esperada: `COMPARACION NO DISPONIBLE`.

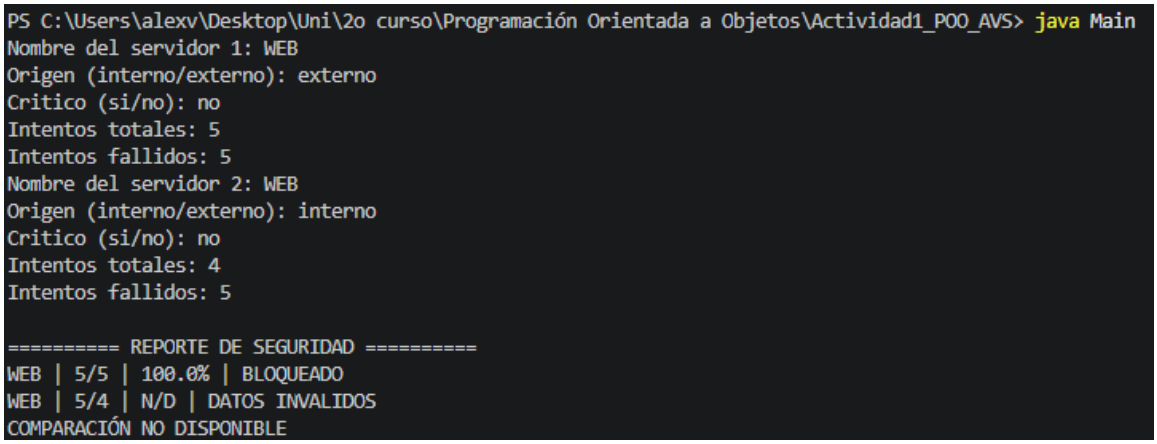


Figura 3: Prueba 3: casos 5 y 6.

3.4. Prueba 4: casos 7 y 8

- Servidor 1: WEB, interno, no, 0 intentos, 0 fallidos. Resultado esperado: `DATOS INVALIDOS`.
- Servidor 2: CORREO, local, no, 10 intentos, 2 fallidos. Resultado esperado: `DATOS INVALIDOS`.
- Comparación esperada: `COMPARACION NO DISPONIBLE`.

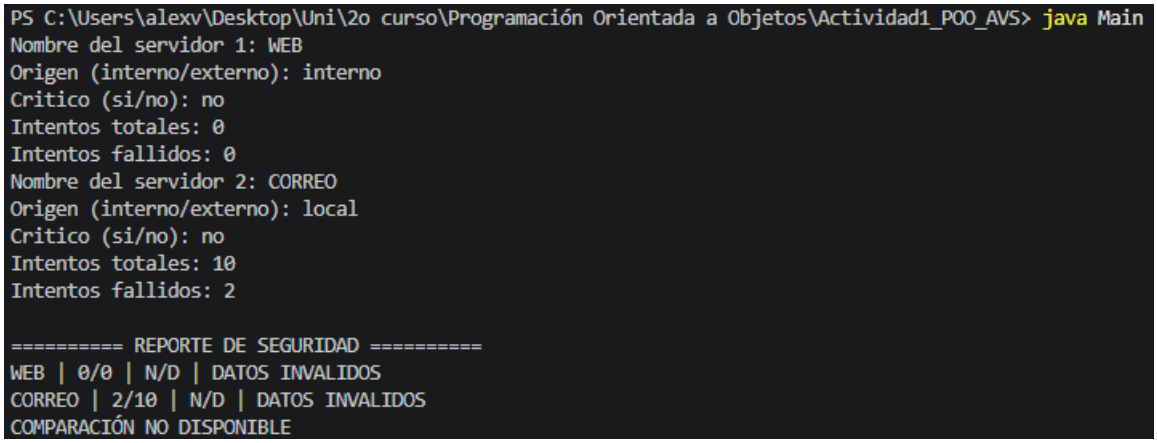


Figura 4: Prueba 4: casos 7 y 8.

3.5. Prueba 5: caso 9 y prueba de espacios

- Servidor 1: WEB, externo, quizás, 10 intentos, 2 fallidos. Resultado esperado: `DATOS INVALIDOS`.
- Servidor 2: VPN, `EXTERNO` , `SI` , 10 intentos, 3 fallidos. Resultado esperado: `BLOQUEADO`.
- Comparación esperada: `COMPARACION NO DISPONIBLE`.

En el segundo servidor he escrito espacios antes y después de EXTERNO y SI para comprobar que el programa los acepta.

```
PS C:\Users\alexv\Desktop\Uni\2o curso\Programación Orientada a Objetos\Actividad1_POO_AVS> java Main
Nombre del servidor 1: WEB
Origen (interno/externo): externo
Critico (si/no): quizás
Intentos totales: 10
Intentos fallidos: 2
Nombre del servidor 2: VPN
Origen (interno/externo):  EXTERNO
Critico (si/no):                    SI
Intentos totales: 10
Intentos fallidos: 3

========== REPORTE DE SEGURIDAD ==========
WEB | 2/10 | N/D | DATOS INVALIDOS
VPN | 3/10 | 30.0% | BLOQUEADO
COMPARACIÓN NO DISPONIBLE
```

Figura 5: Prueba 5: caso 9 y comprobación de espacios y mayúsculas.

4. Preguntas conceptuales

1. ¿Por qué dos objetos de la misma clase pueden almacenar valores diferentes en sus atributos?

Porque cada objeto tiene sus propios atributos. Aunque los dos se creen a partir de la clase `Servidor`, a cada uno se le pueden asignar datos distintos, pues la clase no deja de ser una plantilla.

2. ¿Qué diferencia existe entre `Servidor b = a;` y `Servidor b = new Servidor();`?

Con `Servidor b = a;` las dos variables hacen referencia al mismo objeto. En cambio, con `new Servidor()` se crea un objeto nuevo e independiente.

3. ¿Por qué es importante comprobar `datosValidos()` antes de calcular el porcentaje?

Porque podría haber datos incorrectos o imposibles, como un total de cero intentos. La validación evita calcular porcentajes con datos inválidos y muestra `N/D`, evitando así errores en el programa por operaciones matemáticas imposibles por ejemplo.

5. Conclusión

Con esta actividad he practicado la creación de clases y objetos, el uso de métodos, la lectura de datos con `Scanner` y las condiciones en Java. También he trabajado la validación de entradas y la comparación de los resultados de dos objetos distintos.